Global Headquarters
3307 Hillview Avenue
Palo Alto, CA 94304
Tel: +1 650-846-1000
Toll Free: 1 800-420-8450
Fax: +1 650-846-1005
www.tibco.com

How to Configure Apache Kafka in an AWS Kubernetes Environment

This document describes how to configure Apache Kafka in an Amazon Elastic Container Service for Kubernetes (Amazon EKS).

Version 2.5 July 2020

Document updated for AKD 2.5

For more information, please contact:

TIBCO Software Inc.
3303 Hillview Avenue
Palo Alto, CA 94304
USA

TIBCO®

Table of Contents

TIBCO®

Table of Figures

TIBCO®

1 Overview

Running Apache Kafka on Amazon EKS involves:

- Configuring the Amazon Elastic Container Service for Kubernetes (EKS) for Apache Kafka.
- Configuring the Amazon Elastic Container Registry (ECR) for the Docker® image registry
- Creating a Docker® image embedding Kafka/Zookeeper and hosting it in ECR
- Configuring and creating Kafka/Zookeeper Kubernetes containers based on the Docker image

1.1 Supported Versions

The steps described in this document are supported for the following versions of the products and components involved:

- TIBCO Apache Kafka 2.5.0 is required

  Apache Kafka can be downloaded from either *edelivery.tibco.com* , or https://www.tibco.com/products/tibco-messaging/downloads.
- Docker Community/Enterprise Edition should be most recent version. Docker Community version V19.03.8 was used in conjunction with this document
- Kubernetes 1.15 or *newer*

1.2 Prerequisites

The reader of this document must be familiar with:

- Docker concepts
- Amazon AWS console and the AWS CLI
- Kubernetes concepts, installation, and administration
- Apache Zookeeper/Kafka configuration

1.3 Prepare Local Environment

The following infrastructure should already be in place or *created*:

- A *Linux* or *MacOS* machine equipped for building Docker images
- The following software must already be downloaded to the *Linux* or *macOS* machine equipped for building Docker images.

  Note: All software must be for Linux!

- TIBCO Apache Kafka 2.5.0 has been downloaded
- The *tibakd_eks_files_2.5.zip* (Docker and Kubernetes build files) has been downloaded from *https://community.tibco.com/wiki/tibcor-messaging-article-links-quick-access*

- Create a directory, place tibakd_eks_files_25.zip in the directory. *Unzip* tibakd_eks_files_2.5.zip.
- Unzip the *TIB_msg-akd-core-2.5.0_linux_x86_64.zip*, and place *TIB_msg-akd-core-2.5.0_linux_x86_64.tar.gz* in the newly created *tibakd_eks_files/docker/zookeeper/bin/tar* and the *tibakd_eks_files/docker/kafka/bin/tar* directories. These are required to build the Docker images. The *deb* and *rpm* files are not required, and can be discarded.

1.4 Prepare Preliminary AWS Account and Kubernetes Configuration

Use the following to prepare the preliminary environment to install Kafka on EKS. For more details on preparing AWS, see the Getting started documentation for EKS.

- An AWS account is required. If necessary, create one at http://aws.amazon.com and follow the on-screen instructions.
- Use the region selector in the navigation bar to choose the AWS Region to deploy Kafka in EKS. The documentation will refer to *us-east-1*.

  Note: Currently, not all AWS regions and Availability Zones (AZ) support EKS. As of June 2020, the following AWS regions support EKS:

US	EU	AP
N. Virginia (us-east-1)	Ireland (eu-west-1)	Singapore (ap-southeast-1)
Ohio (us-east-2)	Frankfurt (eu-central-1)	Tokyo (ap-northeast-1)
Oregon (us-west-2)	Stockholm (eu-north-1)	Sydney (ap-southeast-2)
Montreal (ca-central-1)	London (eu-west-2)	Seoul (ap-northeast-2)
Sao Paulo (sa-east-1)	Paris (eu-west-3)	Mumbai (ap-south-1)
	Bahrain (me-south-1)	Hong Kong (ap-east-1)

- Install and configure Amazon AWS CLI on the workstation used.

  Note: After creating the AWS account, ensure the AWS *credential* and *config* files are created, and contain the appropriate AWS *key, secret key, profile,* and *region.*

  Export the following environmental variable: export AWS_SDK_LOAD_CONFIG=1 when the default AIM Role is *not* used.

- Install Docker on the workstation to build the TIBCO Kafka images.
- Install the kubectl command-line tool do manage and deploy applications to Kubernetes in AWS from a workstation.

1.5 Apache Kafka Architecture

.

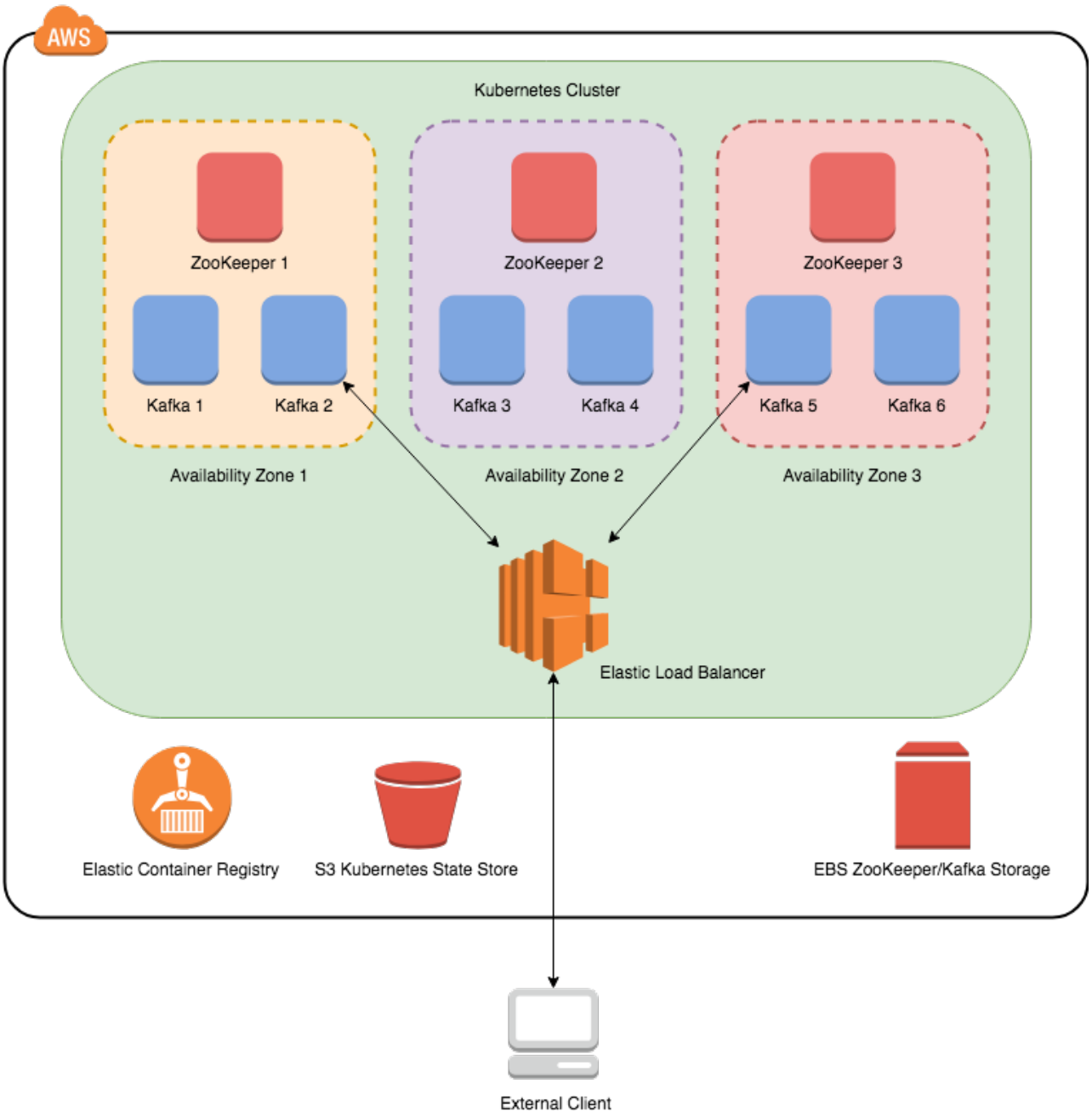

Figure 1 - Kafka Architecture on EKS

This document will outline the creation of the Kafka architecture shown above on Amazon Elastic Container Service for Kubernetes. Using this guide, along with the accompanying software, will create:

- Kubernetes cluster that spans three AWS availability zones.
- Three Zookeeper instances that guarantee high availability of cluster coordinator, each deployed in a different AWS availability zone.

- Six Kafka brokers, two in each AWS availability zone.
- Kafka cluster has been configured with replication factor of three and rack-awareness, which guarantees that each record will be replicated across all three availability zones. Outage of single zone will not interrupt the Kafka service, nor cause data unavailability.
- External access to the Kafka brokers is made available through AWS Elastic Load Balancer.
- Kafka and Zookeeper are configured to use persisted storage. Fast GP2 SSD drives managed by Amazon Elastic Block Store. EBS improves node failover time, because storage from decommissioned node is reattached to new instance. The Kafka broker does not need to copy data from all assigned partitions, because it is already locally available.

  Note: the S3 Kubernetes state store is no longer required.

TIBCO®

2 Preparing the Zookeeper and Kafka Docker Images

This section will outline how to build the Docker images for Zookeeper and Kafka, and how upload the images to the AWS Elastic Container Registry.

2.1 Build Docker images

- Ensure Docker is running on your workstation. Use *docker images* to verify Docker is available.
- Open a terminal shell and navigate to the directory where the *tibakd_eks_files* directory is located. The *docker* and *kubernetes* directories will be underneath, with several subdirectories.
- The *tib_msg-akd-core-2.5.0_linux_x86_64.tar.gz* should already be in the *docker/zookeeper/bin/tar* and the *docker/kafka/bin/tar* directory.
- Navigate to the *docker/zookeeper* directory.
- Execute the **make build** command to build the Zookeeper - Docker image. Alternatively, if you do not have **make** utility installed, use the *run_build.sh* script. The Docker image, *tibco/zookeeper* with the *2.5.0* tag will be created.
- Navigate to the *docker/kafka* directory.
- Execute **make build** command to build the Kafka – Docker image. Alternatively, if you do not have **make** utility installed, use the *run_build.sh* script. The Docker image, *tibco/kafka* with the *2.5.0* tag will be created.
- To test the Docker images, open a second terminal shell, and navigate the *docker/zookeeper* directory. Execute **make run** command to start the Zookeeper – Docker image. In the first terminal shell, and navigate the *docker/kafka* directory. Execute **make run** command to start the Kafka – Docker image. Both should start successfully, and the Kafka broker should connect to Zookeeper.

2.2 Upload Docker images to ECR

The Docker images have to be uploaded to AWS ECR. This section will outline creating the ECR registries and uploading the Docker images to ECR.

- Create two new ECR repositories named **tibco/zookeeper** and **tibco/kafka** in AWS. The repositories can be created via the *AWS CLI* or via the console. Please note the URL of your ECR repository (e.g. *123456789012.dkr.ecr.us-east-1.amazonaws.com*).

```
> aws ecr create-repository --repository-name tibco/zookeeper --
region us-east-1

> aws ecr create-repository --repository-name tibco/kafka – region
us-east-1
```

Figure 2 - Create ECR Repositories

- Retrieve the login command to use to authenticate your Docker client with AWS registry. Adjust AWS region and access keys created in the previous step.

```
> $(aws ecr get-login --no-include-email --region us-east-1)
```

Figure 3 - Get ECR login

- Tag the Docker images to the ECR repository using the URL of the appropriate repository account ID instead of *123456789012* and region.

```
> docker tag tibco/zookeeper:2.5.0 123456789012.dkr.ecr.us-east-
1.amazonaws.com/tibco/zookeeper:latest
> docker tag tibco/kafka:2.5.0 123456789012.dkr.ecr.us-east-
1.amazonaws.com/tibco/kafka:latest
```

Figure 4 - Tag EMS Docker image

- Push the EMS Docker images to ECR. Replace *123456789012* with the appropriate repository account ID and region.

```
> docker push 123456789012.dkr.ecr.us-east-
1.amazonaws.com/tibco/zookeeper:latest

> docker push 123456789012.dkr.ecr.us-east-
1.amazonaws.com/tibco/kafka:latest
```

TIBCO®

3 Create the Kubernetes Cluster on AWS

A new Kubernetes cluster must be created in EKS. There are just a few steps necessary to create the Kubernetes cluster in EKS. Follow the *Getting Started* EKS User Guide to create a new Kubernetes cluster. Cluster can be created via the AWS console or with *eksctl*.

Note: Creating the EKS cluster via *eksctl* is highly recommended. This following will be based on using *eksctl.*

Note: If AWS account has multiple roles or there are multiple clusters, it is recommended that the AWS CLI be used, and the *–profile=XXX* is used to ensure the correct permissions/roles are used.

- Install *eksctl* following the AWS documentation, if not already installed.
- Create the new EKS cluster. The following example shows the minimum inputs. The names, number of nodes, zones, EC2 instance size can all be adjusted to meet the requirements of the environment. It will take several minutes for the cluster to be created.

  Note: Read through the eksctl documentation. There are other parameters which can be used, such as if the nodes should be "managed".

```
eksctl create cluster \
-n akdeks \ (1)
-r us-east-1 \ (2)
--zones us-east-1a,us-east-1b,us-east-1c \ (3)
--nodegroup-name akd-workers \ (4)
-t m5n.2xlarge \ (5)
--nodes 6 \ (6)
--nodes-min 4 \ (7)
--nodes-max 6 \ (8)
--node-volume-size 30 \ (9)
--vpc-cidr x.x.x.x/x (10)
```

Figure 5 - eksctl inputs

(1): The name of the EKS Cluster
(2): The AWS region where the EKS cluster will be built
(3): The AWS Zones. This can be increased to fit requirements. Verify the AZ has the resources first.
(4): The Node Group name for the nodes (workers)
(5): The EC2 instance size. A larger instance can be used to fit the requirements.
Note: If the default size for the instance referenced is 32 GB of RAM and 8 vCPUs. The instance size can be adjusted to meet the requirements. A minimum of 20GB of RAM and 6 vCPUs is required. If adjusted, the resource limits in the *zookeeper* and *kafka* yaml files should considered.
(6): The number of cluster nodes. Six (6) are recommended.
(7): The minimum number of nodes. Four (4) are required.
(8): The maximum number of nodes. This should be at least six (6).

(9): The volume size (GB) of each node. Not used for persisted data. Can be small.
(10): The CIDR of the VPC to use. If not included, eksctl will choose.

- To check on the status of the creation of the cluster, use:

```
> aws eks describe-cluster --name <EKS Cluster> --region <AWS
Region> --query cluster.status
```

- Test the configuration before continuing. Use *kubectl get svc* and *kubectl get nodes.* The results should be similar to the following. **Note:** *Cluster-IP* and *number of nodes* can be different depending on the environment:

```
NAME                                STATUS   ROLES    AGE     VERSION
ip-192-168-13-12.ec2.internal       Ready    <none>   9m36s   v1.16.12-eks-904af05
ip-192-168-49-243.ec2.internal      Ready    <none>   9m35s   v1.16.12-eks-904af05
ip-192-168-51-184.ec2.internal      Ready    <none>   9m36s   v1.16.12-eks-904af05
ip-192-168-65-97.ec2.internal       Ready    <none>   9m30s   v1.16.12-eks-904af05
ip-192-168-8-226.ec2.internal       Ready    <none>   9m37s   v1.16.12-eks-904af05
ip-192-168-82-129.ec2.internal      Ready    <none>   9m32s   v1.16.12-eks-904af05
```

Figure 6 - Kubectl results

If a Kubernetes cluster is not shown, resolve any issues before continuing. If there are issues, it is usually a Kubernetes/AWS permissions/IAM role issue.

4 Deploy Zookeeper and Kafka in EKS

The Zookeeper and Kafka containers can now be deployed in EKS.

With EKS, the *Kubernetes* command line tool, *kubectl,* is used to configure the Kubernetes cluster for Apache Kafka on EKS.

4.1 Create a New Namespace

Create a new *namespace* in Kubernetes, if desired. This is recommended to separate environments in Kubernetes. The *default* namespace can be used.

Note: If the namespace *kafka* is not used, ensure the provided yaml files are modified to use the correct namespace or *default*. The examples shown below will use the *kafka* namespace.

```
> kubectl create namespace kafka
```

Figure 7 - Create kafka namespace

4.2 Update the Zookeeper and Kafka Yaml files

The *zookeeper.yaml, kafka.yaml,* and the *kafka-service.yaml* file will need a modification for the environment. This section will outline the modifications.

4.2.1 Zookeeper

The *zookeeper.yaml* must be modified for the appropriate ECR repository defined in section 2.2.

Find and modify the following line in *zookeeper.yaml* from **<your ECR>** to the ECR used for the Zookeeper container similar as the examples below:

```
image: <your ECR>/zookeeper:latest

image: 123456789012.dkr.ecr.us-east-
1.amazonaws.com/tibco/zookeeper:latest
```

Figure 8 - Zookeeper.yaml modification

No other modifications are required or recommended, unless familiar with Kubernetes configurations.

4.2.2 Kafka

4.2.2.1 Kafka.yaml

The *kafka.yaml* must be modified for the appropriate ECR repository defined in section 2.2.

Find and modify the following line in *kafka.yaml* from **<your ECR>** to the ECR used for the Kafka container similar as the examples below:

```
image: <your ECR>/kafka:latest

image: 123456789012.dkr.ecr.us-east-1.amazonaws.com/tibco/kafka:latest
```

Figure 9 - Kafka.yaml modification

No other modifications are required or recommended, unless familiar with Kubernetes configurations.

4.2.2.2 Kafka-Service.yaml

The kafka-service.yaml file provides the service ports for internally and externally accessing Kafka. The following example shows just the services for *kafka-0* broker. Brokers 1-5 will be similar.

```
apiVersion: v1
kind: Service
metadata:
  name: broker
  namespace: kafka
  annotations:
    service.alpha.kubernetes.io/tolerate-unready-endpoints: "true"
spec:
  ports:
  - port: 9092
    name: client
  clusterIP: None
  selector:
    app: kafka
---
apiVersion: v1
kind: Service
metadata:
  name: kafka-0
  namespace: kafka
  labels:
    app: kafka-0
spec:
  type: LoadBalancer
  ports:
  - port: 32400 (1)
    name: client-external
    targetPort: 9093
    nodePort: 32400 (1)
    protocol: TCP
  selector:
    statefulset.kubernetes.io/pod-name: kafka-0
  loadBalancerSourceRanges:
  - <you trusted IP Range in the form of 0.0.0.0/0> (2)
```

Figure 10 - Kafka-service.yaml example

1) External port to the Kafka-0 Broker. This can be changed to another port number if desired.

2) Set the *trusted IP Range* to the IP range that can access the Kafka Broker externally. 0.0.0.0/0 is open to the world, and is NOT recommended.

4.3 Deploy Zookeeper and Kafka

After the configuration files are updated, Zookeeper and Kafka can be deployed into AWS EKS. Use the following to deploy Zookeeper and Kafka:

- Open command shell and navigate to the *tibakd_eks_files/kubernetes* directory.
- Execute **make build** command to deploy Zookeeper and Kafka. Alternatively, if you do not have **make** utility installed, issue below set of commands manually. Scripts will create new Kubernetes (kafka) namespace, services and dedicated storage class for Zookeeper and Kafka pods.

```
> kubectl apply -f ./namespace.json
> kubectl -n kafka apply -f ./zookeeper/zookeeper-storage.yaml,./zookeeper/zookeeper-service.yaml,./zookeeper/zookeeper.yaml
> kubectl -n kafka apply -f ./kafka/kafka-storage.yaml,./kafka/kafka-service.yaml,./kafka/kafka.yaml
```

Note – It will take several minutes for this step to complete!

- Execute the command below to review state of the cluster. Wait until all Zookeeper and Kafka instances are up and running. Again, this will take some time to complete. It can take ~30 minutes to complete.

```
> kubectl get pods -n kafka
```

- The output from *get pods* will look similar to the following example

```
NAME          READY   STATUS    RESTARTS   AGE
kafka-0       1/1     Running   0          7m22s
kafka-1       1/1     Running   0          7m6s
kafka-2       1/1     Running   0          6m26s
kafka-3       1/1     Running   0          5m43s
kafka-4       1/1     Running   0          5m2s
kafka-5       1/1     Running   0          4m23s
zookeeper-0   1/1     Running   0          18m
zookeeper-1   1/1     Running   0          18m
zookeeper-2   1/1     Running   0          18m
```

Figure 11 - Get Pods example

4.4 Stopping or Deleting the Environment

- To stop the Zookeeper and Kafka processes without deleting them, use the **kubectl scale** operation to set its number of replicas to 0.

For example:

```
> kubectl scale --replicas=0 statefulset kafka -n kafka
> kubectl scale --replicas=0 statefulset zookeeper -n kafka
```

- To start them again, set its number of replicas back to 3 and 6 respectively.

```
> kubectl scale --replicas=3 statefulset zookeeper -n kafka
> kubectl scale --replicas=6 statefulset kafka -n kafka
```

- To delete the environment. open command shell and navigate to the *tibakd_eks_files/kubernetes* directory.
- Execute **make clean** command to delete Zookeeper and Kafka. Alternatively, if you do not have **make** utility installed, issue below set of commands manually. Scripts will delete the services and dedicated storage class for Zookeeper and Kafka pods.

```
>  kubectl -n kafka delete -f ./zookeeper/zookeeper-
storage.yaml,./zookeeper/zookeeper-
service.yaml,./zookeeper/zookeeper.yaml
>  kubectl -n kafka delete -f ./kafka/kafka-
storage.yaml,./kafka/kafka-service.yaml,./kafka/kafka.yaml
```

- Delete the persisted storage using the following:

```
> kubectl delete pv,pvc --all
```

Note: this assumes there are no other pvs or pvcs defined. If there, then the pvs and pvcs must be deleted separately.

TIBCO®

5 Testing the Kafka Environment

In the previous step Zookeeper and Kafka was deployed to a Kubernetes cluster running in the AWS cloud environment. Zookeeper provides only internal access limited to Kubernetes cluster, so we will have to connect to one of the Kafka pods to create a new topic to test with. On the other hand, Kafka brokers expose local, as well as the external service via Amazon ELB.

- Connect to one of Kafka pods and create a new sample topic.

```
> kubectl exec -it kafka-0 -n kafka -- /bin/bash
# kafka-topics.sh --create --zookeeper ${_KAFKA_ZOOKEEPER_CONNECT}
--replication-factor 3 --partitions 100 --topic my-topic --config
min.insync.replicas=2
```

- While still connected to Kafka pod, try to send and receive messages. Please note that we use *broker* Kubernetes service, which is available only internally inside the cluster.

```
# kafka-console-producer.sh --broker-list kafka-
0.broker:9092,kafka-1.broker:9092,kafka-2.broker:9092 --topic my-
topic --request-required-acks all
# kafka-console-consumer.sh --bootstrap-server kafka-0.broker:9092
--from-beginning --topic my-topic
```

- To access Kafka directly from your local workstation, we have to use the Amazon ELB endpoint of at least one Kafka broker. Use the following command to get the external IP and port number assigned to each Kafka broker. Note the external IP and port.

```
> kubectl get services -n kafka -o wide
```

- Send and receive messages from your local workstation. Please note that you do not need to provide the endpoint of every broker, the Kafka client will discover topology of the cluster once it establishes connectivity with any of the nodes. In the following example, *af176121ab0ed11e88cfe060e158e737-819917813.us-east-1.elb.amazonaws.com:32400* is the external IP and port. **Note:** this is assumed that the service port is set to *32400.*

```
> kafka-console-producer.sh --broker-list
af176121ab0ed11e88cfe060e158e737-819917813.us-east-
1.elb.amazonaws.com:32400 --topic my-topic --request-required-acks
all
> kafka-console-consumer.sh --bootstrap-server
af176121ab0ed11e88cfe060e158e737-819917813.us-east-
1.elb.amazonaws.com:32400 --from-beginning --topic my-topic
```